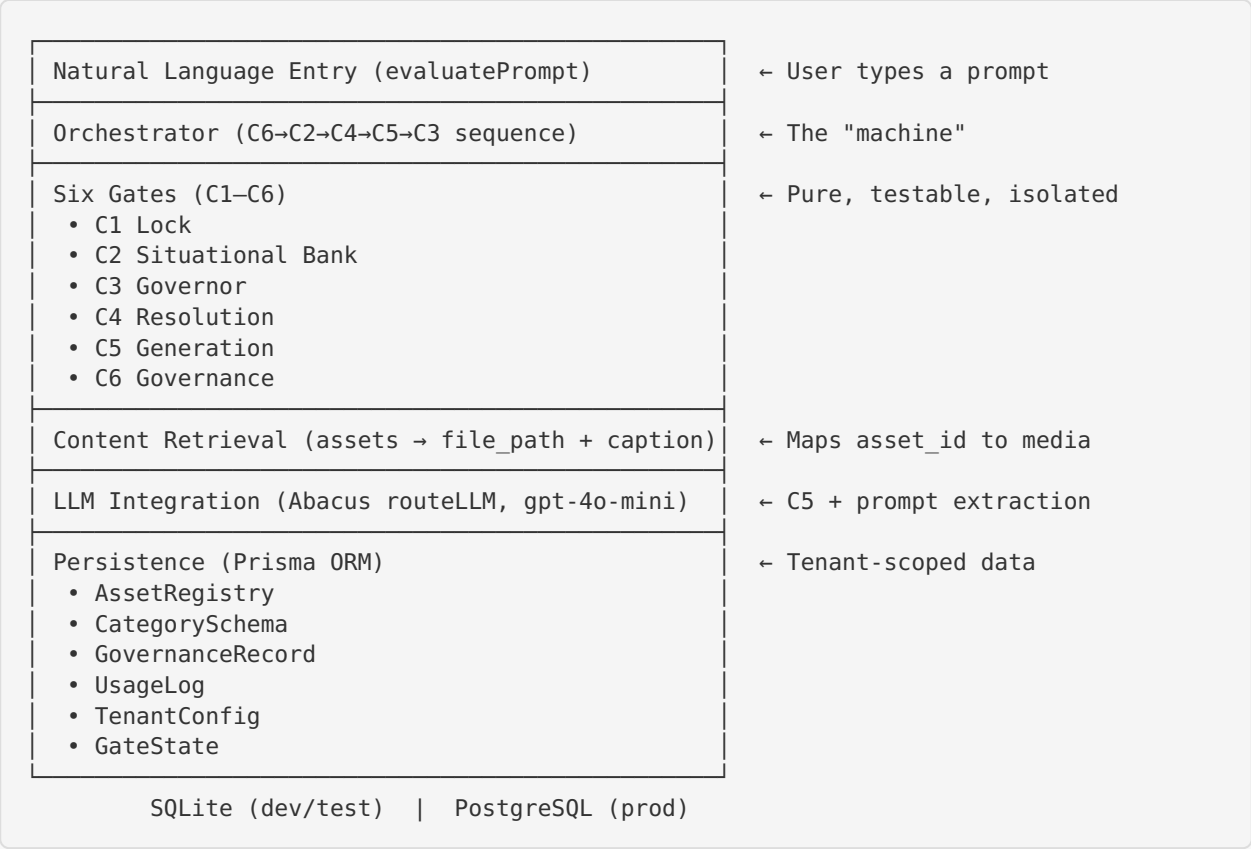


DRCS Systems Ladder

How the engine is built, layer by layer — from persistence to natural language.

The Stack



What's Built vs. What Remains

Layer	Status	Notes
Persistence	✓ Complete	All 7 models (AssetRegistry, CategorySchema, UsageLog, GovernanceRecord, Tenant-Config, GateState, ProposalApprovalLog). Migrations applied.
LLM Integration	✓ Complete	callLLM() operational, Abacus routeLLM verified live, error handling solid.
Content Retrieval	✓ Complete	resolveAsset(), findAsset-ByCategory() tested.
C1 Lock	✓ Complete	4/4 tests passing. Single-use enforcement works.
C2 Situational Bank	✓ Complete	6/6 tests passing. Signal-only matching verified (no calendar path).
C3 Governor	✓ Complete	7/7 tests passing. Rolling cap (3/30-day) enforced. Fail-closed on log integrity.
C4 Resolution	✓ Complete	12/12 tests passing. Caption-first 3-step ladder operational.
C5 Generation	✓ Complete	8/8 tests passing. LLM caption generation working.
C6 Governance	✓ Complete	7/7 tests passing. All four dispositions reachable.
Orchestrator	✓ Complete	Gate sequence (C6→C2→C4→C5→C3) with short-circuiting + audit trail.
Natural Language Entry	✓ Complete	evaluatePrompt() maps user intent → category name → verdict.
Web Console	✓ Complete	Single-page form live at localhost:3000. Prompt input + structured fallback.

Layer	Status	Notes
CLI	✅ Complete	<code>npm run evaluate --prompt "..."</code> working.
C7 Compliance	🔴 Not Designed	No spec, no implementation.
C8 Distribution	🔴 Not Designed	No spec, no implementation.

Build Status: 107 tests passing across 10 test suites. TypeScript build clean. End-to-end verified (prompt → LLM → file path returned).

Integration Map

How a natural language prompt flows through the engine

```

User types: "make a hype post about hitting 10k followers"
↓
evaluatePrompt(tenant_id, prompt)
↓
extractPromptFields:
  - Loads tenant's category names from DB
  - Passes list to LLM: "pick the best match"
  - LLM returns: "Victory"
↓
evaluate() builds structured request:
  trigger: { condition: "Victory" }
  condition_signal: { situation: "Victory" }
  content_need: { caption: "Victory" }
↓
GATE C6 (Governance)
  - disposition: PUBLISH
  - creates GovernanceRecord
↓
GATE C2 (Situational Bank)
  - resolves "Victory" → category_id: "victory"
  - available_assets: ["08_victory_jump"]
↓
GATE C4 (Resolution)
  - Step 1: checks for exact caption match
  - Finds asset with caption "Victory"
  - Returns: EXISTING_AS_IS, asset_id: "08_victory_jump", caption: "Victory"
↓
GATE C3 (Governor)
  - Checks usage log: count = 0 within 30-day window
  - allowed: true
↓
Content Retrieval Layer
  - resolveAsset("08_victory_jump")
  - Returns: { file_path: "media/assets/zilly_mascot.png", caption: "Victory" }
↓
Orchestrator returns verdict:
{
  decision: "PUBLISH",
  outcome: "PUBLISHED",
  file_path: "media/assets/zilly_mascot.png",
  caption: "Victory",
  source: "AS_IS",
  trail: [C6 ✓, C2 ✓, C4 ✓, C3 ✓]
}

```

Total latency (live test): ~2 seconds (includes one LLM call for prompt extraction + DB queries + gate sequence).

Why This Order Matters

C6 first: Governance rejects ideas before they consume resources. If disposition is REJECT or HOLD, the pipeline short-circuits — no category lookup, no content resolution, no LLM generation.

C2 after C6: Only ideas that passed governance get category resolution.

C4 before C5: Caption-first resolution exhausts reuse options before calling the LLM. Minimizes generation costs.

C3 last: Frequency cap is checked only after content is resolved. If C4/C5 found nothing usable, C3 never runs (nothing to cap).

Short-circuiting: If any gate blocks, the orchestrator stops and returns the blocking gate's reason. No wasted downstream work.

Surfaces Entry Points

Surface	File	Entry Function	User
Web Console	<code>src/server/index.ts</code>	<code>POST /api/evaluate-prompt</code>	Human via browser
CLI	<code>src/cli/index.ts</code>	<code>npm run evaluate -- prompt "..."</code>	Human via terminal
Programmatic	<code>src/index.ts</code> exports	<code>evaluate-Prompt(tenant_id, prompt)</code>	Upstream system / API

All three surfaces call the same orchestrator — zero duplicate logic.

What “Complete” Means

A layer is marked **Complete** when:

- ☒ Implementation matches the locked design principle
- ☒ All acceptance criteria verified via tests
- ☒ Build passes (TypeScript clean, no warnings)
- ☒ Integration tested (layer works with layers above/below it)
- ☒ Tenant isolation verified (multi-tenant tests pass)

C1–C6, the orchestrator, and all surfaces meet this bar. C7–C8 don't exist yet.